

NS-RL:Neurosymbolic Reinforcement Learning Based Autonomous UAV Navigation

Snehil Sharma^{**}, Ribhav Singla^{**}, Rijul Tandon^{**},
Peter Vamplew^{*}, Paridhi Naithani^{**}, Sakshi Kaushal^{**}

^{**}University Institute of Engineering & Technology, Panjab University,
Chandigarh, India.

^{*}Federation University, Victoria, Australia.

Contributing authors: snehilsharma0504@gmail.com;
ribhavsingla2166@gmail.com; letscomerijul@gmail.com;
p.vamplew@federation.edu.au; paridhinaithani21@gmail.com;
sakshi@pu.ac.in;

Abstract

Reinforcement learning (RL) has gained significant popularity for autonomous Unmanned Aerial Vehicle (UAV) navigation. In this work, we extend this paradigm through a neurosymbolic reinforcement learning framework that augments a standard RL agent with symbolic navigation rules. The proposed approach integrates human-provided domain knowledge with a Proximal Policy Optimization (PPO) agent trained in a lightweight, custom-built UAV environment based on the MuJoCo physics engine. We evaluate a vanilla PPO agent V-PPO and its neurosymbolic extension (NS-PPO) in environments featuring dynamic maps as well as randomized start-goal configurations. Additionally we train AR-PPO(Augmented reward PPO) with the same symbolic knowledge provided as additional rewards to V-PPO. Experimental results show that NS-PPO achieves faster convergence, reduced exploration time, and higher success rates compared to the baseline V-PPO and AR-PPO, demonstrating the effectiveness of incorporating symbolic reasoning into policy learning for autonomous UAV navigation.

Keywords: Neurosymbolic Reinforcement Learning

1 Introduction

Neuro-symbolic AI aims to unify two perspectives within artificial intelligence: the pattern recognition capabilities of neural networks and the reasoning of symbolic systems [1]. In RL, external knowledge has been incorporated in various forms, including heuristic reward shaping, interactive feedback, imitation learning, transfer learning, and multi-source knowledge aggregation [2]. These methods do not directly guide to a lower level of action mappings, Neurosymbolic RL diverges from these methods by introducing explicit symbolic priors that persist throughout training and execution. These prior rules over-write RL agent actions and forces it to execute expert action when Rule is available. That reduces the burden of RL agent to determine best action at that state and trusts completely on expert rule. Although this may create a drawback that if the rule is not globally correct then it will block RL agent learning, however we have shown a global optimal rule which improves the learning drastically.

Acharya et al. [3] describe neurosymbolic reinforcement learning systems using three conceptual paradigms: learning for reasoning, reasoning for learning, and learning-reasoning. The approach adopted in this work aligns with the reasoning-for-learning paradigm, wherein we use reasoning based knowledge to develop symbolic rules which eventually aid in Learning. Despite strong theoretical motivation, neurosymbolic RL remains underexplored in the context of UAV navigation. we develop a lightweight yet physically realistic UAV navigation environment constructed using the MuJoCo physics engine. A key novel feature of our platform is a human-playable interface, which allows human experts to perform the navigation task in the same map as the agent. This design makes it possible to directly compare autonomous agents with human strategies, (decision at each step, overall accuracy, timesteps). To our knowledge, such a direct comparison between human and autonomous UAV control has not been previously examined in the literature. We present comparison of all three agents with human expert. We leverage LiDAR to detect obstacles in our map. we have also extended the work to Vision-based navigation using mujoco in built camera imagery and Open Cv based pre trained model, the results are provided in Appendix B. To promote reproducibility and support further research, we release our complete codebase at [Github](#).

We train three PPO agents 1) V-PPO (Vanilla PPO) which is the standard PPO with basic reward structure, 2) NS-PPO (Neurosymbolic PPO) which has symbolic rules integrated with its policy and 3) AR-PPO (Augmented Reward PPO) with a particular bias between V-PPO and NS-PPO and NS-PPO has extra information and knowledge about environment which allows to perform better anyways.

Contributions. The major contributions of this work are as follows:

- We introduce a neurosymbolic reinforcement learning formulation that integrates rule-based reasoning directly into policy learning for UAV navigation.
- We develop a lightweight, MuJoCo-based UAV simulation environment leveraging LiDAR based navigation, extending it to a human-playable game interface, enabling direct comparison between human and agent performance.

- We conduct extensive experiments across LiDAR-based perception to analyze the effectiveness of symbolic guidance, with vision-based results available in the appendix.
- We release a complete, open-source research framework that enables plug-and-play experimentation and web-based **Deployment** without requiring local installation.

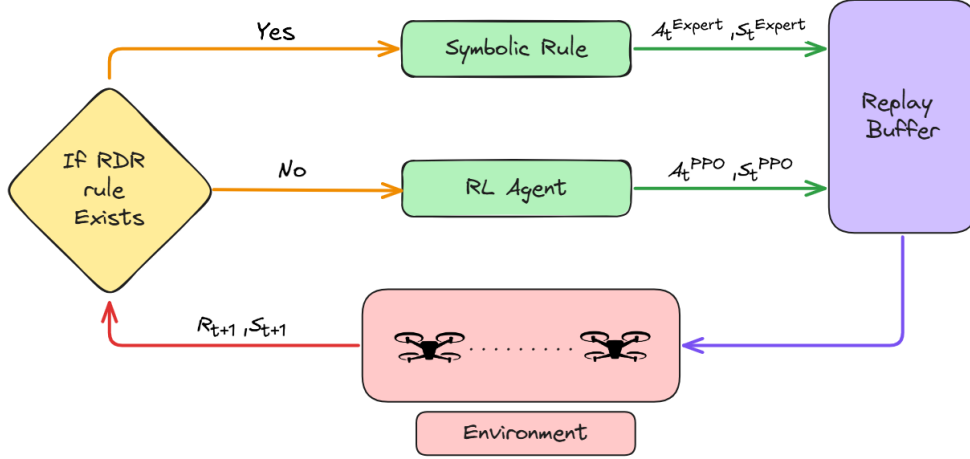


Fig. 1: NS-RL Framework Architecture

2 Related Work

Neuro-symbolic AI is the combination of representational and learning capacity of neural networks with the interpretability and logical structure of symbolic reasoning. Nawaz et al. [1] provide a comprehensive overview, outlining key challenges and highlighting the promise of neurosymbolic systems for achieving improved generalization and explainability. De Bot et al. [4] introduced a framework for constructing MiniHack environments in which neurosymbolic RL agents employ probabilistic logic shields to ensure safe, interpretable and constrained behavior. According to recent research, neurosymbolic RL may be divided into three major paradigms: learning for reasoning, reasoning for learning, and learning-reasoning, where the learning process is guided and shaped by symbolic priors [3]. Our method is most in line with the *reasoning-for-learning* paradigm, in which policy optimisation is directly influenced by symbolic structures. A taxonomy for Assisted Reinforcement Learning (ARL) is developed by Bignold et al. [2], describing the representation, retention, and utilisation of external information. Heuristic RL, learning from demonstration, transfer learning, and multi-source knowledge integration are all included in their architecture, which offers a modular basis for creating RL systems that include symbolic or human-informed

priors. Bignold et al. [5] introduced a mechanism for retaining symbolic rules provided through interaction, enabling agents to generalize advice into persistent constraints that continuously influence policy learning. Hussonnois et al. [6] proposed the Controlled Diversity with Preference (CDP) framework, wherein explicit reward shaping has been replaced with expert-driven preference models. This shifts skill discovery toward behavior regions aligned with expert intent, improving both interpretability and task alignment. Amador et al. [7] introduce symDQN, a neurosymbolic extension of Dueling DQN that incorporates symbolic reasoning layers. Their formulation improves reward efficiency and yields policies with clearer interpretability, demonstrating the value of embedding symbolic priors directly within deep RL networks. Gutierrez et al. [8] developed NEUSIS, a compositional neurosymbolic framework for UAV search tasks. This system fuses multimodal perception, probabilistic world modeling, and symbolic planning to achieve structured and constraint-aware flight behavior. Guan et al. [9] describe symbolic model incompleteness and its consequences for directing RL policy search, formalising the relationship between symbolic models and underlying MDPs.

In contrast to above approaches, we show the simplest method to integrate symbolic rules with a RL without any one-hot encoding or embeddings to use the symbolic rules.

The proposed rule-based policy layer overwrites RL agent action selection when symbolic conditions are met, providing a structured and interpretable mechanism that continuously shapes policy optimization during learning.

3 Methodology

We propose the NS-RL framework, a general approach for augmenting any RL algorithm with symbolic knowledge through a rule-based decision hierarchy as shown in 1. We instantiate this framework with PPO to create NS-PPO and model the UAV navigation problem as a MDP (Markov Decision Process), defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$. Here, $\mathcal{S}$ represents the state space, $\mathcal{A}$ is the action space, $\mathcal{T}$ is the transition probability function, $\mathcal{R}$ is the reward function, and γ is the discount factor. The UAV operates in an $8\text{m} \times 8\text{m}$ bounded planar environment. We investigate the NS-RL framework using LIDAR-based sensing modality with range sensors for obstacle awareness. For vision-based detection results using camera imagery, please refer to Appendix B.

For the LIDAR-based variant, the continuous state space is defined in Equation 1

$$\mathbf{s}_t = \{\mathbf{p}_t, \mathbf{v}_t, \mathbf{l}_t, g_t\} \in \mathbb{R}^{29} \quad (1)$$

where $\mathbf{s}_t$ denotes the state vector at timestep t , $\mathbf{p}_t$ is the UAV position (x, y, z), $\mathbf{v}_t$ is the UAV velocity (vx, vy, vz), $\mathbf{l}_t$ comprises the LIDAR readings from 16 rays, and g_t encodes the goal distance and direction. The LIDAR sensor, with 16 rays distributed over 360 and a maximum range of 2.9m, provides raw range measurements from which we extract engineered features to form $\mathbf{l}_t$. This representation helps the UAV to detect obstacles.

3.1 Environment Components

The action space is continuous representing normalized velocity changes: $\mathbf{a}_t = [v_x, v_y, v_z]^T \in [-1, 1]^3$, where these actions are scaled to physical velocities and smoothed to ensure realistic UAV dynamics. The total reward at timestep t is given by Equation 2:

$$R_t = r_p + r_l + r_c + r_b \quad (2)$$

where r_p is the progress-toward-goal reward, r_l is the per-step penalty, r_c is the collision-risk penalty, and r_b is the boundary-proximity penalty defined as follows:

$$r_p = 10.0 \times \Delta d_{\text{goal}} \quad (3)$$

where Δd_{goal} denotes the change in distance to goal (negative when moving closer).

$$r_l = -0.1 \text{ per step} \quad (4)$$

$$r_c = \begin{cases} -5.0 & \text{if } d_{\text{obstacle}} < 0.3 \text{ m (danger zone)} \\ -1.0 & \text{if } 0.3 \text{ m} \leq d_{\text{obstacle}} < 0.5 \text{ m (warn zone)} \\ 0.0 & \text{otherwise} \end{cases} \quad (5)$$

where d_{obstacle} is the minimum LIDAR-detected obstacle distance.

$$r_b = \begin{cases} -1.0 & \text{if } d_b < 0.8 \text{ m} \\ 0.0 & \text{otherwise} \end{cases} \quad (6)$$

where d_b denotes the distance to nearest environment boundary. For AR-PPO only, an additional LIDAR-based goal reward is used:

$$r_{lg} = \begin{cases} 15.0 \times \max(0, \cos(\theta)) & \text{if goal detected by LIDAR} \\ 0.0 & \text{otherwise} \end{cases} \quad (7)$$

where θ denotes the angle between UAV velocity and goal direction. The lesser the angle, higher will be the value for $\cos(\theta)$ motivating UAV to move towards the goal. Terminal rewards are applied upon episode termination: goal achievement yields +100, collision results in -100, and out-of-bounds violations yield -100. The simulation environment is implemented using the MuJoCo physics engine (version 3.3.6) with a control timestep of $\Delta t = 0.05\text{s}$. Obstacles are randomly distributed using grid-based placement with perturbations to ensure diversity in obstacle configurations. To facilitate sample-efficient learning, we employ a curriculum learning strategy that progressively increases the task difficulty. The curriculum systematically varies the obstacle count as shown in Equation 18:

$$n_{\text{obs}} \in \{1, 2, \dots, 10\} \quad (8)$$

where n_{obs} denotes the obstacle count in each curriculum stage. This approach allows the agent to learn foundational skills in simpler environments before tackling more complex scenarios, For each map we run 50 seeds , so overall 500 training episodes .

3.2 Implementation of NS-PPO

The NS-RL framework is instantiated with PPO as follows. The objective is to learn a policy $\pi : \mathcal{S} \mapsto \mathcal{A}$ that maximizes the expected cumulative discounted reward. Our framework incorporates a Ripple Down Rules (RDR) system to encode expert knowledge [10]. At each timestep, the system evaluates rule activation through mathematical checks: Rule 1 activates when line-of-sight is verified ($\min(\text{obs_dist}_i) \geq d_{\text{goal}}$), and Rule 2 activates when the UAV approaches boundaries ($d_{\text{boundary}} < 0.8$ m). These conditions are checked in priority order before defaulting to PPO. An RDR is defined as a tuple in Equation 9:

$$\mathcal{R}_i = (C_i, A_i, E_i) \quad (9)$$

where $\mathcal{R}_i$ denotes the i -th rule in the RDR system, C_i is the condition predicate that determines when the rule applies, A_i is the action mapping that prescribes the action if the condition is true, and E_i is the set of exception rules that override this rule. We implement three key rules in a hierarchical exception-based structure:

Rule R0 (Default Rule): The default rule always applies and returns control to the PPO agent. When no specific symbolic rule is triggered, the PPO agent outputs the action directly:

$$\mathbf{a}_t^{s_0} = \mathbf{a}_t^{\text{PPO}} \quad (10)$$

where $\mathbf{a}_t^{s_0}$ denotes the action from the default symbolic rule, and $\mathbf{a}_t^{\text{PPO}} \in [-1, 1]^3$ is the continuous action sampled from the PPO policy network.

Symbolic Rule 1 (Clear Path to Goal): When the direction towards the goal is clear (no obstacles blocking the line-of-sight), this rule increases the velocity towards the goal, enabling aggressive movement toward the target. The rule employs LIDAR-based line-of-sight verification with angular safety margins. Given 16 LIDAR rays distributed over 360, each ray covers 22.5. Rule 1 checks three rays: the goal-directed ray and its two neighbors (± 22.5 angular buffer), ensuring a clear corridor of approximately 67.5 toward the goal. The activation condition is:

$$C_1(\mathbf{s}_t) = \begin{cases} \text{True} & \text{if } \mathbf{l}_t[\text{goal_ray}] > 0.4 \text{ and neighbors clear} \\ \text{False} & \text{otherwise} \end{cases} \quad (11)$$

where $C_1(\mathbf{s}_t)$ denotes the condition for Rule 1, $\mathbf{l}_t[\text{goal_ray}]$ is the LIDAR reading in the goal direction (normalized to $[0, 1]$ where 0.4 corresponds to approximately 1.16 m clearance given $R_{\text{LIDAR}} = 2.9$ m), and neighbors are adjacent LIDAR rays (± 1 index) that must satisfy $\mathbf{l}_t[\text{neighbor}] > 0.3$. Additionally, the minimum obstacle distance across all rays must exceed the goal distance:

$$\min_{i \in [1, 16]} (\mathbf{l}_t[i] \times R_{\text{LIDAR}}) \geq \|\mathbf{p}_{\text{goal}} - \mathbf{p}_t\|_2 \quad (12)$$

where $R_{\text{LIDAR}} = 2.9$ m is the maximum LIDAR range. When both conditions (Equations 11 and 12) are satisfied, the action increases velocity toward the goal:

$$\mathbf{a}_t^{\text{R1}} = \text{clip}(\mathbf{v}_t^{\text{xy}} + 0.3 \times \mathbf{d}_{\text{goal}}, [-1, 1]^2) \quad (13)$$

where $\mathbf{a}_t^{\text{R1}}$ denotes the action from Rule 1, $\mathbf{v}_t^{\text{xy}} = [v_x, v_y]^T$ is the current 2D vel., $\mathbf{d}_{\text{goal}} = \frac{\mathbf{p}_{\text{goal}} - \mathbf{p}_t}{\|\mathbf{p}_{\text{goal}} - \mathbf{p}_t\|_2}$ is the norm. direction toward goal, 0.3 is the vel. scaling factor, and $\text{clip}(\cdot, [-1, 1]^2)$ clamps to valid bounds.

Symbolic Rule 2 (Boundary Safety): When the UAV approaches the environment boundaries, this rule decreases velocity and directs the UAV away from the boundary to maintain safety. The rule establishes a safety buffer zone of 0.8 m from all four boundaries of the 8×8 m environment. The activation condition requires computing distances to each boundary:

$$d_{\text{west}} = x_t + \frac{W}{2}, \quad d_{\text{east}} = \frac{W}{2} - x_t, \quad d_{\text{north}} = \frac{W}{2} - y_t, \quad d_{\text{south}} = y_t + \frac{W}{2} \quad (14)$$

where $\mathbf{p}_t = [x_t, y_t, z_t]$ is the current UAV position and $W = 8.0$ m is the environment size. The minimum distance determines activation:

$$C_2(\mathbf{s}_t) = \begin{cases} \text{True} & \text{if } d_{\text{boundary}} < 0.8 \text{ m} \\ \text{False} & \text{otherwise} \end{cases} \quad (15)$$

where $d_{\text{boundary}} = \min(d_{\text{west}}, d_{\text{east}}, d_{\text{north}}, d_{\text{south}})$. This creates a safety margin representing 10% of the environment width on each side, preventing boundary collisions while maintaining navigable space. When triggered, the action reduces velocity and escapes the boundary:

$$\mathbf{a}_t^{\text{R2}} = \text{clip} \left(\max(\|\mathbf{v}^{\text{escape}}\|_2, 0.5) \times \frac{\mathbf{v}_t^{\text{xy}} - 0.25 \times \mathbf{d}_{\text{escape}}}{\|\mathbf{v}_t^{\text{xy}} - 0.25 \times \mathbf{d}_{\text{escape}}\|_2 + \epsilon}, [-1, 1]^2 \right) \quad (16)$$

where $\mathbf{a}_t^{\text{R2}}$ denotes the action from Rule 2, $\mathbf{d}_{\text{escape}}$ is the norm. direction away from nearest boundary, 0.25 is the vel. reduction factor, 0.5 is the min. speed threshold, and $\epsilon = 10^{-8}$ prevents division by zero.

where $\mathbf{d}_{\text{escape}}$ is the normalized direction away from the nearest boundary, 0.25 is the velocity reduction factor, 0.5 is the minimum speed threshold, and $\epsilon = 1 \times 10^{-8}$ is a small constant to prevent division by zero.

The evaluation procedure traverses the rule hierarchy in order: first checking Symbolic Rule 1, then Symbolic Rule 2, and finally defaulting to R0. When a rule condition evaluates to true, the corresponding prescribed action is executed, and the procedure terminates (no further rules are evaluated). This exception-based hierarchy ensures that more specific rules override general ones, providing interpretable decision-making.

Our neural network architecture employs a shared feature extractor that processes the 29-dimensional state vector through three fully connected hidden layers of 256 units each, with layer normalization and tanh activation functions. This shared representation is then fed to separate actor and critic heads. The actor head outputs the mean and standard deviation of a Gaussian distribution for continuous action selection, while the critic head produces a scalar value estimate for the current state. This architecture enables efficient parameter sharing while maintaining specialized outputs for policy and value function learning.

The key innovation of our approach lies in the hybrid decision mechanism illustrated in Figure 1. At runtime, the NS-PPO agent implements a rule-first hierarchy that evaluates conditions and selects the most specific applicable rule:

$$\mathbf{a}_t^{\text{NS}} = \begin{cases} \mathbf{a}_t^{\text{R1}} & \text{if } C_1(\mathbf{s}_t) = \text{True (Equation 11)} \\ \mathbf{a}_t^{\text{R2}} & \text{if } C_2(\mathbf{s}_t) = \text{True (Equation 15)} \\ \mathbf{a}_t^{\text{R0}} & \text{otherwise (Equation 10)} \end{cases} \quad (17)$$

where $\mathbf{a}_t^{\text{NS}}$ denotes the final action from the NS-PPO agent at timestep t , rules are checked in priority order $\text{R1} \rightarrow \text{R2} \rightarrow \text{R0}$ (default), and specific rule actions are executed immediately when their conditions are satisfied.

When a specific rule (R1 or R2) condition is satisfied, its prescribed action (Equations 13 or 16) is executed immediately, bypassing the neural network. This provides interpretable, domain-expert-guided behavior in well-understood scenarios. When no specific rule applies, the PPO agent takes control with full autonomy. This hybrid approach allows symbolic knowledge to accelerate learning in specific situations while preserving the flexibility of learned policies for complex, unanticipated scenarios.

To facilitate sample-efficient learning, we employ a curriculum learning strategy that progressively increases the task difficulty. The curriculum systematically varies the obstacle count as shown in Equation 18:

$$n_{\text{obs}} \in \{1, 2, \dots, 10\} \quad (18)$$

with each stage training the agent for 100 episodes. This approach allows the agent to learn foundational skills in simpler environments before tackling more complex scenarios.

4 Experimental Setup

The experimental framework is designed to ensure a fair comparison between the three approaches: vanilla PPO (V-PPO), neurosymbolic PPO (NS-PPO), and auxiliary reward PPO (AR-PPO). All experiments are conducted on **LIDAR-based** environments. Vision-based experimental results are provided in Appendix B.

All agents operate in identical environments with identical primary reward structures. The NS-PPO agent receives additional information in the form of symbolic rules, encoded as direct state-action mappings. To ensure a fair comparison, AR-PPO receives equivalent information through additional reward signals that correspond to the symbolic rules.

For our training configuration, we employ curriculum learning across 10 stages, where stage k trains the agent with exactly k obstacles ($k \in \{1, 2, \dots, 10\}$). We train on maps from 1 to 10 obstacles, using 50 random seeds for each map to ensure robust learning across diverse environmental configurations. For testing, we evaluate our agents on maps with obstacles ranging from 1 to 15, using 10 independent seeds for each obstacle count to assess generalization capability beyond the training distribution. The training hyperparameters are held constant for all agents to ensure a fair comparison, as summarised in Table 1. Specifically, we use a learning rate of

$\alpha_{\text{actor}} = 5 \times 10^{-5}$ for the actor network and $\alpha_{\text{critic}} = 2 \times 10^{-4}$ for the critic network. The policy is updated over $K = 11$ epochs per batch of 1024 transitions, with a discount factor $\gamma = 0.999$ and an entropy coefficient $\beta = 0.01$. In terms of network architecture, all agents use identical actor-critic models with three hidden layers of 256 dimensions each, followed by layer normalization and tanh activation functions. To evaluate performance, we report key metrics for each obstacle count defined as Equation 19:

$$\text{Success Rate} = \frac{\text{Number of episodes reaching goal}}{|\text{Total episodes}|} \quad (19)$$

where Success Rate denotes the proportion of episodes where the goal is reached, and Total episodes is the number of test episodes per obstacle level.

Environment Parameters			
Parameter	Value	Parameter	Value
Env. Size	8×8 m	UAV Height	1.0 m
Timestep (Δt)	0.05 s	LIDAR Rays	16
LIDAR Range	2.9 m	LIDAR Res.	22.5
State Dim.	29	Action Dim.	3
Action Range	$[-1, 1]^3$	Max Velocity	1.0 m/s
Training Parameters			
Parameter	Value	Parameter	Value
Actor LR	5×10^{-5}	Critic LR	2×10^{-4}
Discount γ	0.999	Entropy β	0.01
PPO Clip ϵ	0.1	Epochs K	11
Batch Size	1024	Hidden Dim.	256
Hidden Layers	3	Total Params	~ 264 K
Curriculum & Reward Parameters			
Parameter	Value	Parameter	Value
Training Obstacles	$\{1, \dots, 10\}$	Training Seeds	50 per map
Testing Obstacles	$\{1, \dots, 15\}$	Testing Seeds	10 per map
Curriculum Learning	Yes	Total Training	500 episodes
Progress Reward	$10.0 \times \Delta d$	Living Penalty	$-0.1/\text{step}$
Collision (danger)	-5.0 if $d < 0.3$ m	Collision (warn)	-1.0 if $0.3-0.5$ m
Boundary Penalty	-1.0 if $d < 0.8$ m	LIDAR Goal	up to 15.0
Goal Achievement	+100	Collision/OOB	-100
Goal Threshold	0.5 m	Max Episode Len	50000
Neurosymbolic Parameters			
Parameter	Value	Parameter	Value
Rule Tree Depth	3	LOS Angle Margin	5
Safety Distance	0.5 m	Boundary Safety Dist	0.8 m
Velocity Factor	0.5	Decision Mech.	Binary

Table 1: Comprehensive parameter table listing hyperparameters, environment settings, reward values, and neurosymbolic parameters.

5 Results

We trained all agents on 10 different obstacle configurations, ranging from 1 to 10 obstacles per map, and then evaluated them on maps ranging from 1 to 15 obstacles. For each configuration, we conducted 10 independent runs with different random seeds to ensure statistical robustness. A comprehensive comparison of all agents across different obstacle counts is detailed in Table ??, where we measure the average episode length required to reach the goal and the binary success indicator.

Level	V-PPO		NS-PPO		AR-PPO	
	Success	Avg Steps	Success	Avg Steps	Success	Avg Steps
1	60%	2230	100%	1353	60%	729
2	40%	1052	100%	1877	70%	575
3	50%	941	100%	2295	70%	533
4	40%	7777	100%	2157	50%	3626
5	60%	3469	100%	1996	40%	3159
6	70%	2250	100%	2282	50%	8306
7	60%	1018	100%	1470	40%	640
8	50%	4954	100%	1928	40%	1596
9	40%	1168	100%	1859	60%	1490
10	20%	474	80%	1869	50%	1966
11	50%	2166	100%	2471	40%	544
12	40%	6008	80%	2416	20%	802
13	30%	3998	100%	2671	30%	1141
14	60%	1616	80%	2217	30%	721
15	40%	1426	70%	3493	20%	684

Table 2: LIDAR-based: Detailed comparison of Pure Neural, Neurosymbolic, and AR-PPO across difficulty levels (1-15)

Metric	V-PPO	NS-PPO	AR-PPO
Success Rate	47.3% (71/150)	94.0% (141/150)	44.7% (67/150)
Avg Steps (success)	2658	2128	1834
Collisions	8	9	16
Out of Bounds	0	0	0

Table 3: LIDAR-based: Overall performance metrics (All 150 episodes aggregated)

5.1 Comparative Analysis Plots

The following figures present comprehensive visualizations of the agent performance across all difficulty levels:

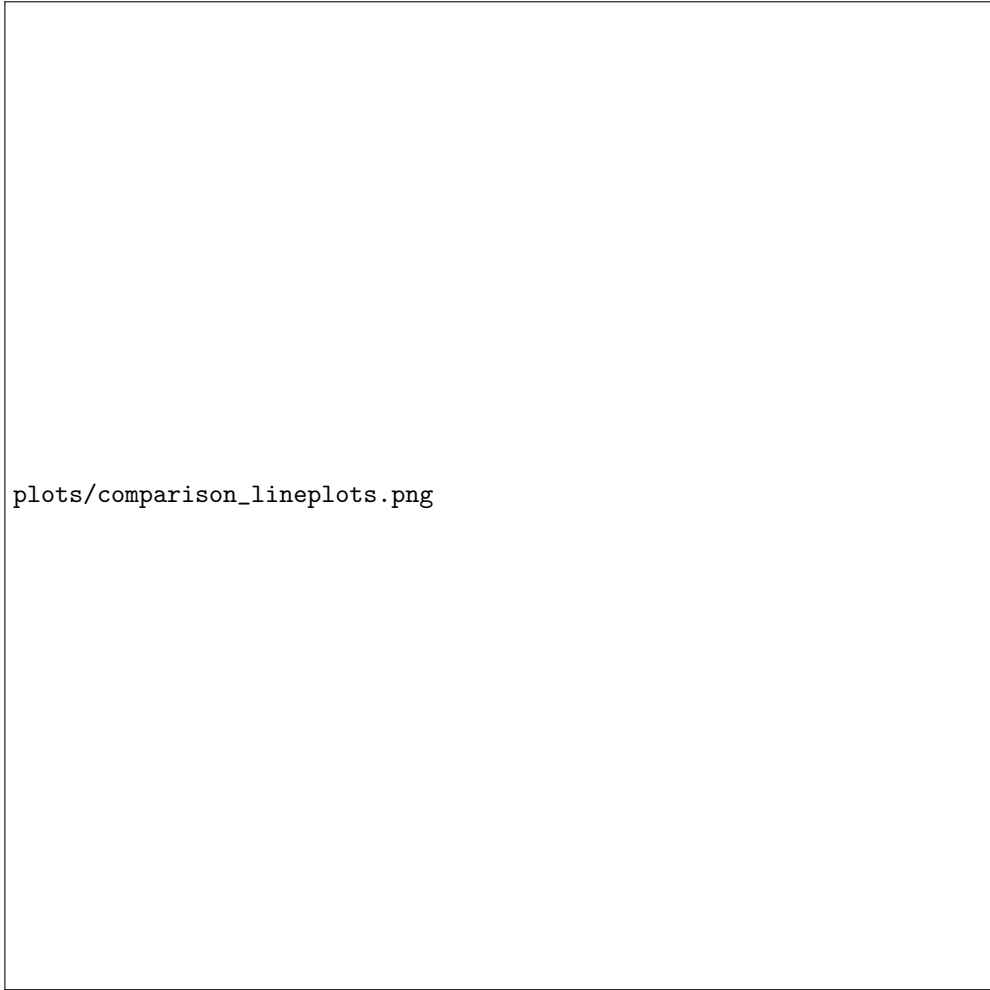


Fig. 2: LIDAR-based performance metrics: (left) Average steps vs difficulty level showing all three agents' efficiency, (right) Success rate vs difficulty level demonstrating Neurosymbolic's superior performance across all levels, with 94% overall success rate compared to Pure Neural (47.3%) and AR-PPO (44.7%).

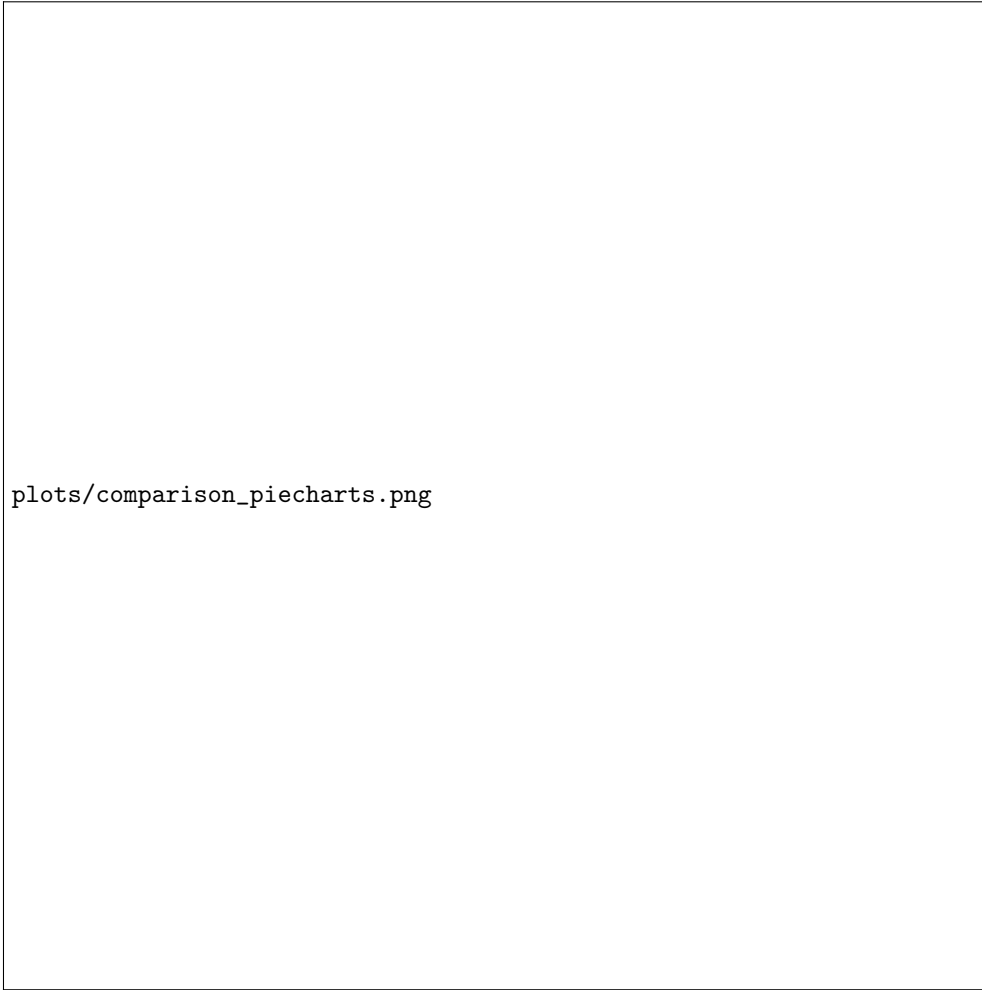
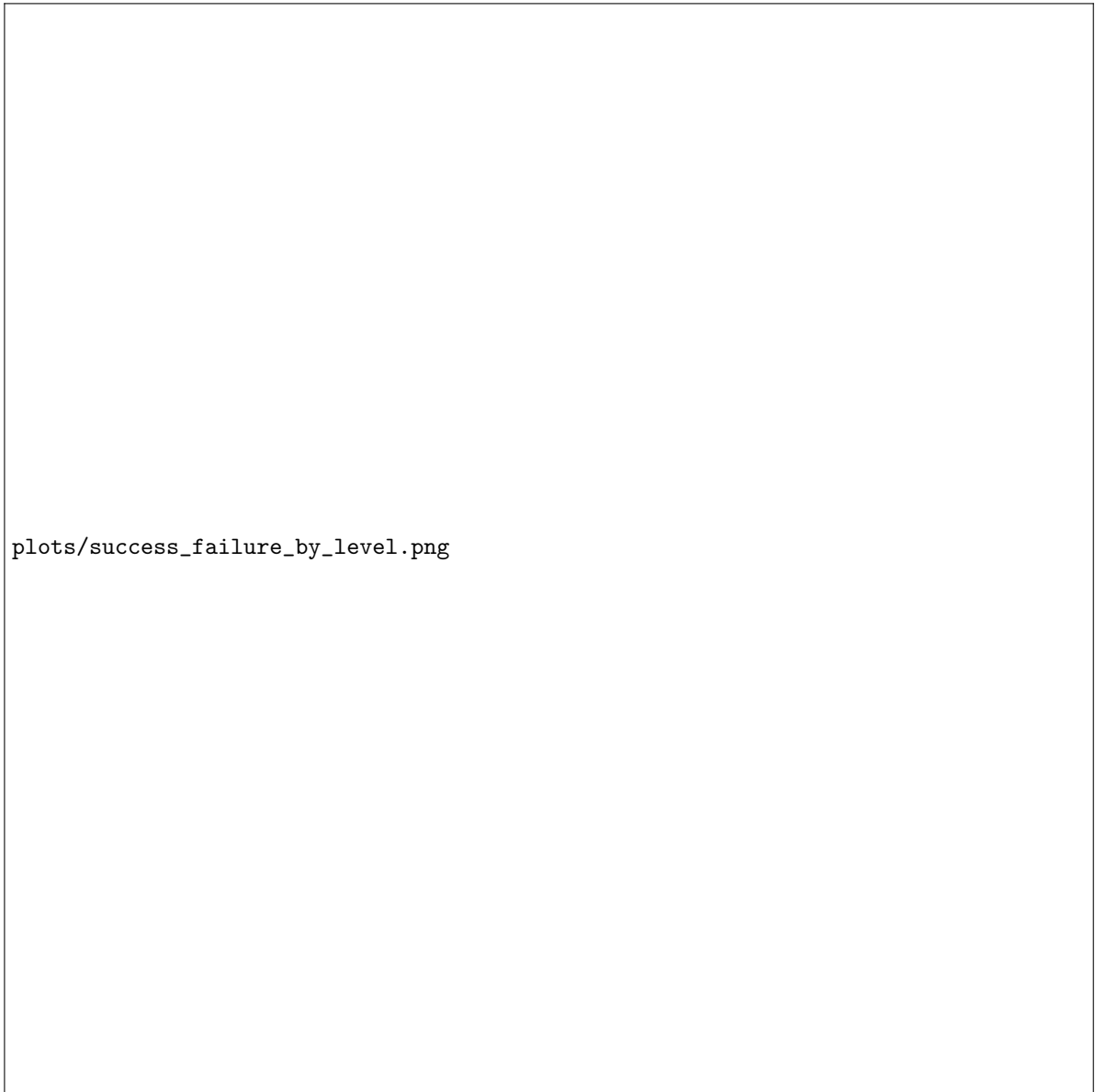


Fig. 3: LIDAR-based success/failure distribution: Pie charts showing the overall success rates across all 150 episodes for each model. Neurosymbolic achieves 94% success rate, significantly outperforming Pure Neural (47.3%) and AR-PPO (44.7%).



plots/success_failure_by_level.png

Fig. 4: Performance by difficulty level: Stacked bar chart showing success and failure counts for each model across all 15 difficulty levels.

6 Conclusion

This research extends neurosymbolic reinforcement learning to UAV navigation, integrating symbolic rules, where conditions directly map to actions. This approach enhances the explainability of RL agent decisions during the learning process. The empirical evaluation shows the performance benefits of symbolic rule integration in three distinct configurations: vanilla PPO, neurosymbolic PPO (NS-PPO), and augmented reward PPO (AR-PPO) using LIDAR-based perception modality.

Future research directions include: investigating higher-dimensional rule hierarchies and their computational trade-offs, conducting detailed timing comparisons between vanilla PPO and NS-PPO policies, incorporating stochastic environmental factors such as wind that introduce realistic dynamics to UAV trajectories, and optimizing path planning to account for battery constraints alongside navigation efficiency. These extensions will further advance the practical applicability of neurosymbolic approaches in autonomous aerial systems.

Appendix A Deployment

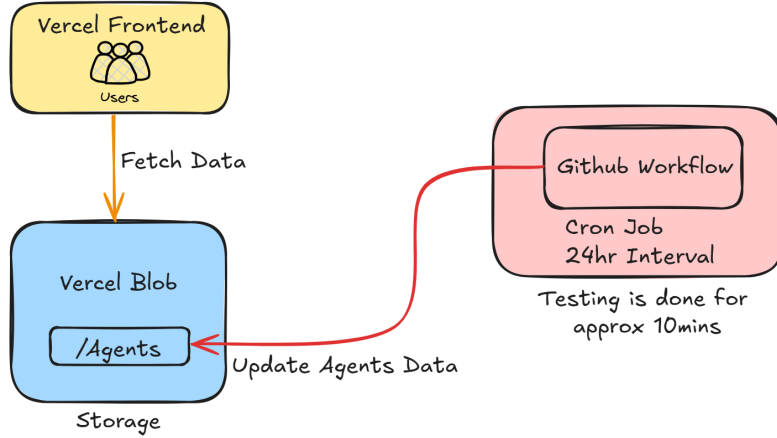


Fig. A1: Deployment Architecture

The system architecture integrates an interactive frontend with an automated data-generation backend to deliver daily-updated UAV simulation results. The frontend, built using JavaScript, HTML, CSS, MuJoCo, and Three.js, allows users to visualize how the UAV navigates from the start point to the goal using three different agent models. Instead of storing static results, the frontend fetches dynamic simulation data from Vercel Blob Storage, where the most recent performance metrics of the agents are maintained. The testing and data-generation process is handled in a separate GitHub repository that contains the evaluation pipeline. A GitHub Workflow, scheduled via a 24-hour cron job, automatically runs the `uav_test_comparison_new.py` script to evaluate

the agents against different obstacle levels. After generating the latest performance data, the workflow uploads the updated results directly to the /Agents directory in Vercel Blob Storage. This ensures that users always access freshly computed, real-world simulation outcomes rather than repeated or static map configurations, thereby enhancing the reliability and relevance of the visualization experience.

Appendix B Vision-Based Obstacle Detection and Experimental Results

B.1 Vision-Based Environment

For the vision-based variant, the state space incorporates camera observations instead of LIDAR. The UAV receives RGB images from an onboard camera, which are processed through a convolutional neural network to extract visual features. The resulting state representation captures spatial information about obstacles and the goal from camera imagery, enabling vision-based navigation without explicit range measurements.

B.2 Vision-Based Obstacle Detection Results

We evaluated all three agents (V-PPO, NS-PPO, and AR-PPO) using vision-based obstacle detection on the same 10 different obstacle configurations. A comprehensive comparison of the agents across different obstacle counts is detailed in the tables below.

Obs	Expert Steps	Vanilla Steps	AR-PPO Steps	NS-PPO Steps	Expert Success	Vanilla Success	AR-PPO Success	NS-PPO Success
1	-	-	-	-	-	-	-	-
2	-	-	-	-	-	-	-	-
3	-	-	-	-	-	-	-	-
4	-	-	-	-	-	-	-	-
5	-	-	-	-	-	-	-	-
6	-	-	-	-	-	-	-	-
7	-	-	-	-	-	-	-	-
8	-	-	-	-	-	-	-	-
9	-	-	-	-	-	-	-	-
10	-	-	-	-	-	-	-	-

Table B1: Vision-based: RL agent comparison (Vanilla PPO, AR-PPO, NS-PPO) with human expert

Metric	Vanilla PPO	AR-PPO	NS-PPO
Success Rate	-	-	-
Collision Rate	-	-	-
Timeout Rate	-	-	-

Table B2: Vision-based: Overall performance metrics (Vanilla PPO, AR-PPO, NS-PPO)

Obs 2-7	Avg Steps			Success Rate		
	Vanilla	AR-PPO	NS-PPO	Vanilla	AR-PPO	NS-PPO
1	-	-	-	-	-	-
2	-	-	-	-	-	-
3	-	-	-	-	-	-
4	-	-	-	-	-	-
5	-	-	-	-	-	-
6	-	-	-	-	-	-
7	-	-	-	-	-	-
8	-	-	-	-	-	-
9	-	-	-	-	-	-
10	-	-	-	-	-	-

Table B3: Vision-based: Vanilla PPO vs AR-PPO vs NS-PPO comparison

Acknowledgments

The authors acknowledge the use of large language models (LLMs) to assist in writing portions of this manuscript and developing code implementations. All technical content, experimental results, and scientific contributions were conceived, executed, and validated by the authors.

Funding

The authors did not receive support from any organization for the submitted work.

Conflict of Interest

The authors have no competing interests to declare that are relevant to the content of this article.

References

- [1] Nawaz, U., Anees-ur-Rahaman, M., Saeed, Z.: A review of neuro-symbolic ai integrating reasoning and learning for advanced cognitive systems. *Intelligent Systems with Applications*, 200541 (2025)
- [2] Bignold, A., Cruz, F., Taylor, M.E., Brys, T., Dazeley, R., Vamplew, P., Foale, C.: A conceptual framework for externally-influenced agents: An assisted reinforcement learning review. *Journal of Ambient Intelligence and Humanized Computing* **14**(4), 3621–3644 (2023)
- [3] Acharya, K., Raza, W., Dourado, C., Velasquez, A., Song, H.H.: Neurosymbolic reinforcement learning and planning: A survey. *IEEE Transactions on Artificial Intelligence* **5**(5), 1939–1953 (2023)
- [4] Debot, D., Venturato, G., Marra, G., De Raedt, L.: Neurosymbolic reinforcement learning: Playing minihack with probabilistic logic shields. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, pp. 29631–29633 (2025)

- [5] Bignold, A., Cruz, F., Dazeley, R., Vamplew, P., Foale, C.: Persistent rule-based interactive reinforcement learning. *Neural Computing and Applications* **35**(32), 23411–23428 (2023)
- [6] Hussonnois, M., Karimpanal, T.G., Jha, M.S., Rana, S.: Human-informed skill discovery: Controlled diversity with preference in reinforcement learning. *Expert Systems with Applications*, 128604 (2025)
- [7] Amador, I., Gierasimczuk, N.: SymDQN: Symbolic knowledge and reasoning in neural network-based reinforcement learning. *arXiv preprint arXiv:2504.02654* (2025)
- [8] Gutierrez, J., Cai, Z., Rojas Cardenas, C., Leo, K., Zhang, C., Backman, K., Li, H., Li, B., Ghorbanali, M., Datta, S., et al.: NEUSIS: a compositional neuro-symbolic framework for autonomous perception, reasoning, and planning in complex UAV search missions
- [9] Guan, L., Sreedharan, S., Kambhampati, S.: Leveraging approximate symbolic models for reinforcement learning via skill diversity. In: *International Conference on Machine Learning*, pp. 7949–7967 (2022). PMLR
- [10] Compton, P., Kang, B.H.: *Ripple-down Rules: the Alternative to Machine Learning*. Chapman and Hall/CRC, ??? (2021)